

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-a4f40a18b014aae01.jsonl`  
- SHA-256: `ed2b4e3ed8f9947ac4a6c44ae6985949a5e734aac01ed2d82bdb84aa0694b514`  
- Source modified: `2026-06-13T19:15:51+00:00`  
- Imported at: `2026-07-08T16:00:00+00:00`  
- project: `wf_095035a0-e9f`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-13T19:14:49.684Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

```
FINDING: {"title":"Two different 'min_players' ? persisted in_court_ratio always uses the  
literal default 2, ignoring config  
fusion.min_players", "severity":"low", "file":"backend/pipeline/segmenters/fusion_hybrid.py:106,  
backend/pipeline/detectors/fusion_features.py:99,200,208", "detail":"There are two distinct  
'min_players': (a) the served Gate-B threshold `FusionConfig.min_players` (used only in  
`_select_for_handoff`), and (b) the `in_court_ratio` parameter (default 2) baked into  
`compute_person_features`. `_person_features` calls `ff.compute_person_features(...)` WITHOUT  
forwarding any min_players, so the persisted `in_court_ratio` feature is always computed at the  
hardcoded threshold 2, regardless of `indexing.fusion.min_players`. This is defensible (the  
persisted feature is meant to be a fixed, config-independent column for the learned fusion in  
P3), and the gate threshold legitimately lives elsewhere ? but the name collision invites a  
future reader to assume changing `fusion.min_players` also retunes the persisted  
`in_court_ratio`, which it does not. No functional bug today.", "fix":"Optional: add a one-line  
comment at fusion_hybrid.py:106 noting that in_court_ratio's threshold is intentionally the  
fixed feature-column default (2) and is independent of the served Gate-B `fusion.min_players`,  
so the two are not conflated later."}
```

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:14:52.541Z)

I'll verify this finding by reading the actual code. Let me start by examining the relevant files.

3. user (2026-06-13T19:14:53.508Z)

```
1      """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).  
2  
3      Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
```

```

4     TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5     do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6     shuttle-seeded window via two overridable seams the engine exposes:
7
8     - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9       GPU-free, computed from the trajectory we already have); and, only when the person cue
10      is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11      (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12      fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
13      never imports torch / touches the GPU.
14      - ``_select_for_handoff`` ? optional served Gate A/B: when
15      ``indexing.fusion.min_crossings``
16      (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
17      are
18      dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19      **Persisted candidates remain the full superset** regardless, so the offline
20      experiment
21      harness can re-score any variant.
22
23      Defaults reproduce the substrate exactly: net_crossings/person features are persisted
24      but
25      nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
26      seeds
27      and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
28      default
29      segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
30      through the
31      experiment harness (EXPERIMENT_HARNESS.md), never silently.
32      """
33      from typing import Any, Dict, List, Optional, Tuple
34
35      from backend.pipeline.segmenters.base import segmenter_registry
36      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
37      from backend.pipeline.detectors.base import DetectorRunner
38      from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
39      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
40      from backend.pipeline.detectors import rally_gate
41      from backend.pipeline.detectors import fusion_features as ff
42
43      class FusionSegmenter(TrajectoryHybridSegmenter):
44          """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
45          B)."""
46
47          family = "fusion"
48          display_label = "Fusion"
49
50          def __init__(self, db: Any, config: Any):
51              super().__init__(db, config)
52              # Built lazily only if the person cue is enabled (keeps torch/GPU off the
53              default path).
54              self._person: Optional[Any] = None
55
56          @property
57          def name(self) -> str:
58              return "fusion_hybrid"
59
60          @property
61          def description(self) -> str:
62              return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
63              rallies, "
64
65                  "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
66              and the "
67
68                  "AI provider sets semantic boundaries.")

```

```

58     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
59         substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
60         if substrate == "tracknet":
61             cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
62             return TrackNetRunner(cfg), {
63                 "weights_path": cfg.weights_path,
64                 "queue_length": cfg.queue_length,
65                 "fusion_substrate": "tracknet",
66             }
67         wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
68         return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
69
70     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
71                                frame_width: float, video_path: str,
72                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
73         stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
74         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
75         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
76         # over the whole trajectory by rally_gate (camera-agnostic).
77         gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
min_crossings=0)
78         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
79
80         fcfg = idx_cfg.get("fusion", {})
81         if fcfg.get("cue_flags", {}).get("person", False):
82             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
83         return stats
84
85     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
86                          frame_width: float, video_path: str,
87                          fcfg: Dict[str, Any]) -> Dict[str, float]:
88         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
89         frame range and compute the scale/translation-invariant person features. Lazily
builds
90         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
91         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
92         from backend.pipeline.detectors.person_detector import PersonDetector
93         if self._person is None:
94             self._person = PersonDetector(self.config)
95
96         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
97         start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)
98         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
99         fw = det["frame_width"] or frame_width
100        fh = det["frame_height"] or 0.0
101
102        poly = fcfg.get("court_polygon")
103        polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
104        net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
105        shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
106        return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
107                                         court_polygon=polygon, net=net)
108
109     def _select_for_handoff(self, windows: List[Dict[str, Any]],
110                             window_stats: List[Dict[str, Any]],
111                             idx_cfg: Dict[str, Any]) -> List[int]:

```

```

112         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
113         (0) keep every window ? identical to the substrate. Persisted candidates are
114         untouched (full superset)."""
115         fcfg = idx_cfg.get("fusion", {})
116         min_crossings = int(fcfg.get("min_crossings", 0))
117         min_players = int(fcfg.get("min_players", 0))
118         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120         keep: List[int] = []
121         for i, st in enumerate(window_stats):
122             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                 continue
124             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                 continue
126             keep.append(i)
127         return keep
128
129
130     segmenter_registry.register(FusionSegmenter)
131

```

4. user (2026-06-13T19:14:53.648Z)

```

1     """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3     NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4     math is unit-testable in a GPU-free dev container. The detection (person boxes,
5     shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6     module only turns those signals into a handful of **scale/translation-invariant**
7     features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8     memorise this court/camera and fail to generalise off a small labelled set).
9
10    Conventions:
11    - A person box is ``(x1, y1, x2, y2)`` in **pixels** ; ``(track_id, box)`` when motion
needs it.
12    - ``per_frame`` is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video
frame
13        index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
timing).
14    - A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
trajectory shape.
15    - ``court_polygon`` is normalised 0-1 ``(x, y), ...`` or ``None`` (None ? no mask,
every
16        detection counts ? the player-count-only mode).
17    - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
0.0).
18
19    Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20    floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
21    """
22    from __future__ import annotations
23
24    from typing import Dict, List, Optional, Sequence, Tuple
25
26    BBox = Tuple[float, float, float, float]
27    Net = Tuple[str, float]
28
29
30    def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31        """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32        (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""

```

```

33     if polygon is None or len(polygon) < 3:
34         return False
35     x, y = point
36     inside = False
37     n = len(polygon)
38     j = n - 1
39     for i in range(n):
40         xi, yi = polygon[i]
41         xj, yj = polygon[j]
42         # Does the horizontal ray at y cross edge (i, j)?
43         if (yi > y) != (yj > y):
44             x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45             if x < x_cross:
46                 inside = not inside
47         j = i
48     return inside
49
50
51 def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52     """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53     the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54     if frame_width <= 0 or frame_height <= 0:
55         return (0.0, 0.0)
56     x1, _y1, x2, y2 = box
57     return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60 def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61     if frame_width <= 0 or frame_height <= 0:
62         return (0.0, 0.0)
63     x1, y1, x2, y2 = box
64     return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67 def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                    court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69     """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70     True (count everyone) when it is None."""
71     if court_polygon is None:
72         return True
73     return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
74
75
76 def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
77                    frame_height: float,
78                    court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
79     """Per-frame count of in-court persons (one entry per sampled frame)."""
80     return [
81         sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
82 court_polygon))
83         for _frame_idx, dets in sorted(per_frame.items())
84     ]
85
86 def _median(vals: Sequence[float]) -> float:
87     if not vals:
88         return 0.0
89     s = sorted(vals)
90     n = len(s)
91     return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
92
93

```

```

94     def players_in_court(counts: Sequence[int]) -> float:
95         """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
96         return _median(list(counts))
97
98
99     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
100         """Fraction of sampled frames with >= `min_players` in court (track stability vs
101 flicker)."""
102         if not counts:
103             return 0.0
104         return sum(1 for c in counts if c >= min_players) / len(counts)
105
106     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
107                           frame_width: float, frame_height: float) -> float:
108         """Mean per-track centre displacement between consecutive sampled frames, normalised
109 by the frame width (resolution-invariant). 0.0 if <2 frames or no shared tracks."""
110         if frame_width <= 0 or len(per_frame) < 2:
111             return 0.0
112         frames = sorted(per_frame.keys())
113         steps: List[float] = []
114         for a, b in zip(frames, frames[1:]):
115             prev = {tid: box for tid, box in per_frame[a]}
116             for tid, box in per_frame[b]:
117                 if tid in prev:
118                     cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
119                     cx1, cy1 = _center_norm(box, frame_width, frame_height)
120                     steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
121         if not steps:
122             return 0.0
123         return sum(steps) / len(steps)
124
125
126     def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
127                         frame_height: float, net: Optional[Net],
128                         court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
129         """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
130
131 A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
132 The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
133
134 """
135         if net is None or not per_frame:
136             return 0.0
137         axis, pos = net
138         both = 0
139         for _frame_idx, dets in per_frame.items():
140             near, far = False, False
141             for _tid, box in dets:
142                 if not person_in_court(box, frame_width, frame_height, court_polygon):
143                     continue
144                 fx, fy = _foot_norm(box, frame_width, frame_height)
145                 coord = fx if axis == "x" else fy
146                 if coord < pos:
147                     near = True
148                 else:
149                     far = True
150             if near and far:
151                 both += 1
152         return both / len(per_frame)
153
154     def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
155 float,
156                        pad_frac: float) -> bool:

```

```

156     """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
157     width/height (the 'adjacent / in-hand' tolerance)?"""
158     x1, y1, x2, y2 = box
159     px = pad_frac * frame_width
160     py = pad_frac * frame_height
161     return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
162
163
164     def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
165                                     frame_width: float, frame_height: float,
166                                     pad_frac: float = 0.03) -> float:
167         """Fraction of (visible shuttle frames that were also person-sampled) in which the
168         shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
169         ~0**
170         ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
171         """
172         if not per_frame or frame_width <= 0 or frame_height <= 0:
173             return 0.0
174         aligned = 0
175         colocated = 0
176         for p in shuttle_points or []:
177             if not getattr(p, "visible", False):
178                 continue
179             dets = per_frame.get(int(p.frame))
180             if dets is None: # shuttle frame wasn't person-sampled ? can't judge
181                 continue
182             aligned += 1
183             if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
184                     for _tid, box in dets):
185                 colocated += 1
186         return colocated / aligned if aligned else 0.0
187
188     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
189     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
190     PERSON_FEATURE_NAMES = [
191         "players_in_court", "two_sided_motion", "mean_player_motion",
192         "in_court_ratio", "shuttle_player_colocation",
193     ]
194
195
196     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
197                                     frame_width: float, frame_height: float,
198                                     court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
199                                     net: Optional[Net] = None,
200                                     min_players: int = 2) -> Dict[str, float]:
201         """Aggregate the per-window person features from per-frame detections + the in-
window
202         shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
203         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
204         return {
205             "players_in_court": players_in_court(counts),
206             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
207             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
208             "in_court_ratio": in_court_ratio(counts, min_players),
209             "shuttle_player_colocation": shuttle_player_colocation(
210                 per_frame, shuttle_points, frame_width, frame_height),
211         }
212

```

Both my hypotheses need checking. Let me verify the key claims in the finding by looking at the config model and the parent class to understand the relationship between the two `min\_players`.

6. user (2026-06-13T19:15:00.705Z)

```
diff --git a/backend/config/models.py b/backend/config/models.py
index bb56881..7f8191c 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -75,6 +75,39 @@ class WasbIndexConfig(BaseModel):
     min_free_gb: float = 0.0

+class FusionConfig(BaseModel):
+    """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
+
+    Every default reproduces control behaviour: the fusion segmenter persists the
+    shuttle net-crossing count (Gate A signal) and ? only when `cue_flags['person']` is
+    true ? the person-cue features, but it does NOT gate the served handoff unless a
+    threshold is raised (`min_crossings` / `min_players` default 0 = OFF). The court mask
+    is optional: `court_polygon=None` ? Gate B counts every detection (player-count-only);
+    supplied ? off-court persons (spectators / adjacent courts) are excluded.
+
+    # Which trajectory substrate seeds the windows (shuttle stays primary).
+    substrate: Literal["wasb", "tracknet"] = "wasb"
+    # Windowing velocity threshold for the fusion family (the engine reads
+    # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
+    # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
+    velocity_threshold: float = 0.02
+    # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
+    # enabled ? so the default path never imports torch / touches the GPU.
+    cue_flags: dict[str, bool] = Field(default_factory=dict)
+    # Served Gate A: drop windows whose shuttle net-crossings < this from the AI handoff.
+    # 0 = OFF (no gating; persisted candidates are always the full superset for offline re-
+    scoring).
+    min_crossings: int = Field(default=0, ge=0)
+    # Served Gate B: when the person cue is on, drop windows with median in-court players
+    # < this. 0 = OFF.
+    min_players: int = Field(default=0, ge=0)
+    # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
+    court_polygon: Optional[list[list[float]]] = None
+    # Net line for the person two-sided-motion split (person features only ? the shuttle
+    # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
+    net_axis: Literal["x", "y"] = "y"
+    net_position: float = Field(default=0.5, ge=0.0, le=1.0)
+
+class IndexingConfig(BaseModel):
+    default_segmenter: str = "yolo_hybrid"
+    use_gpu: bool = True
@@ -106,6 +139,7 @@ class IndexingConfig(BaseModel):

     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
+    fusion: FusionConfig = Field(default_factory=FusionConfig)

     @field_validator("default_segmenter")
     @classmethod
```

7. user (2026-06-13T19:15:01.230Z)

```
docs\CODE_MAP.md:148:[Omitted long matching line]
backend\config\models.py:84:     threshold is raised (`min_crossings` / `min_players` default 0
= OFF). The court mask
backend\config\models.py:102:     min_players: int = Field(default=0, ge=0)
```

```

tests\test_config.py:35:    assert f.min_players == 0                # Gate B OFF
backend\eval\experiment.py:91:    - ``min_players`` ? decision-gate **Gate B** (the future
person cue): keep windows
backend\eval\experiment.py:109:    min_players: int = 0
backend\eval\experiment.py:126:        "min_players": self.min_players,
backend\eval\experiment.py:197:    if variant.min_players > 0:
backend\eval\experiment.py:199:        mask = [m and (col[i] >= variant.min_players) for i, m in
enumerate(mask)]
tests\test_experiment.py:82:                                min_crossings=0, min_players=0)
tests\test_experiment.py:107:    preds = predict(Variant(name="gateB", min_players=2), vc)
backend\pipeline\segmenters\fusion_hybrid.py:15:    (and, with the person cue, ``min_players``)
are raised above 0, sub-threshold windows are
backend\pipeline\segmenters\fusion_hybrid.py:117:        min_players =
int(fcfg.get("min_players", 0))
backend\pipeline\segmenters\fusion_hybrid.py:124:                if person_on and min_players > 0
and st.get("players_in_court", 0.0) < min_players:
tests\test_fusion_features.py:60:    assert ff.in_court_ratio(counts, min_players=2) ==
pytest.approx(0.25)
tests\test_fusion_features.py:61:    assert ff.in_court_ratio(counts, min_players=1) ==
pytest.approx(0.75)
backend\pipeline\detectors\fusion_features.py:99: def in_court_ratio(counts: Sequence[int],
min_players: int = 2) -> float:
backend\pipeline\detectors\fusion_features.py:100:     """Fraction of sampled frames with >=
`min_players` in court (track stability vs flicker)."""
backend\pipeline\detectors\fusion_features.py:103:     return sum(1 for c in counts if c >=
min_players) / len(counts)
backend\pipeline\detectors\fusion_features.py:200:                                min_players: int =
2) -> Dict[str, float]:
backend\pipeline\detectors\fusion_features.py:208:        "in_court_ratio":
in_court_ratio(counts, min_players),

```

8. assistant (2026-06-13T19:15:15.225Z)

Now I have the full picture. Let me verify the key claims of the finding precisely.

The finding states:

1. There are two distinct `min\_players`: (a) the served Gate-B threshold `FusionConfig.min\_players` (used in `\_select\_for\_handoff`), and (b) the `in\_court\_ratio` parameter (default 2) in `compute\_person\_features`.
2. `\_person\_features` calls `ff.compute\_person\_features(...)` WITHOUT forwarding any `min\_players`, so the persisted `in\_court\_ratio` always uses the hardcoded default 2.

Let me verify each precisely against the code.

****Claim 1 ? two distinct min_players:****

- `FusionConfig.min\_players` (config/models.py:102, default 0) ? used in `\_select\_for\_handoff` at fusion_hybrid.py:117 and :124. CONFIRMED. Note: Gate B at line :124 compares against `players\_in\_court`, NOT `in\_court\_ratio`.

- `in\_court\_ratio(counts, min\_players=2)` (fusion_features.py:99) and `compute\_person\_features(..., min\_players=2)` (fusion_features.py:200). CONFIRMED ? default 2.

****Claim 2 ? `\_person\_features` does not forward `min\_players`:****

Looking at fusion_hybrid.py:106-107:

```

```python
return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
 court_polygon=polygon, net=net)
...

```

It passes `court\_polygon` and `net` but NOT `min\_players`. So `compute\_person\_features` uses its default `min\_players=2`, which flows to `in\_court\_ratio(counts, 2)`. CONFIRMED ? the persisted `in\_court\_ratio` is always computed at threshold 2, independent of `FusionConfig.min\_players`.

Now let me adversarially test whether this is actually a **\*\*bug\*\*** vs. a benign name-collision/documentation nitpick, which is what the finding itself concedes ("No functional bug today", severity "low").

Key question: Does the served Gate B (`select\_for\_handoff`) ever use `in\_court\_ratio`? Looking at line :124, Gate B gates on `players\_in\_court`, not `in\_court\_ratio`. So `FusionConfig.min\_players` controls the `players\_in\_court` threshold for the \*served\* gate, while `in\_court\_ratio` is a separate persisted feature column whose own internal `min\_players=2` is a fixed feature definition. These genuinely are independent by design ? the persisted feature is meant to be a config-independent column for the learned fusion (P3).

Let me confirm `in\_court\_ratio` is not consumed anywhere in the served gating path or experiment harness in a way that would make the collision a real bug.

#### 9. user (2026-06-13T19:15:15.817Z)

```
docs\EXPERIMENT_HARNESS.md:175: `in_court_ratio`, `shuttle_player_colocation`; shuttle-state:
`net_crossings`, `direction_reversals`;
docs\MULTI_SIGNAL_FUSION_PLAN.md:138:| `in_court_ratio` | fraction of frames with ?2 in-court
players | track stability vs flicker |
docs\QUALITY_ITERATIONS.md:66:mean_player_motion, in_court_ratio, shuttle_player_colocation`)
persisted into candidate `stats` for P3's
tests\test_fusion_features.py:60: assert ff.in_court_ratio(counts, min_players=2) ==
pytest.approx(0.25)
tests\test_fusion_features.py:61: assert ff.in_court_ratio(counts, min_players=1) ==
pytest.approx(0.75)
tests\test_fusion_features.py:65: assert ff.in_court_ratio([]) == 0.0
backend\pipeline\detectors\fusion_features.py:99:def in_court_ratio(counts: Sequence[int],
min_players: int = 2) -> float:
backend\pipeline\detectors\fusion_features.py:192: "in_court_ratio",
"shuttle_player_colocation",
backend\pipeline\detectors\fusion_features.py:208: "in_court_ratio":
in_court_ratio(counts, min_players),
backend\pipeline\validators\relevance.py:54: min_court_ratio =
rel_cfg.get("court_pixels_min_ratio", sport_rel_cfg.get("court_pixels_min_ratio", 0.05))
backend\pipeline\validators\relevance.py:93: "green_threshold": min_court_ratio
backend\pipeline\validators\relevance.py:97: if avg_green_ratio < min_court_ratio:
backend\pipeline\validators\relevance.py:101: message=f"Video rejected: Relevance
check failed. The scene does not contain a recognizable {sport_name} court (court area found:
{round(avg_green_ratio * 100, 2)}% vs min required {round(min_court_ratio * 100, 2)}%).",
```

#### 10. assistant (2026-06-13T19:15:20.526Z)

This confirms the design intent. The documentation (MULTI\_SIGNAL\_FUSION\_PLAN.md:138) explicitly defines `in\_court\_ratio` as "fraction of frames with ?2 in-court players" ? i.e., the threshold of 2 is the \*definition\* of this feature column, not a tunable. It is consumed only as a persisted feature for P3's learned fusion, never in the served gating path (which uses `players\_in\_court`, line :124).

Let me check the experiment harness column mapping to be fully sure `in\_court\_ratio` isn't tied to `min\_players` there.

#### 11. user (2026-06-13T19:15:21.407Z)

```
85 `segmenter_registry` + `IndexingConfig` do the actual selection in production).
86 New cues are recorded here default-OFF.
87 - ``min_crossings`` ? decision-gate **Gate A** : keep only windows whose persisted
88 ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
89 `rally_gate.py` expressed as a re-scoring knob (kills service-feed / held-
shuttle
90 windows ? see `MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
91 - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
92 whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
93 cue persists that feature).
94 - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
95 `compare()` scores videos with the SHIPPED model as-is (no per-fold training);
96 required for `compare_unlabeled` on a learned variant.
97 - ``feature_names`` ? the persisted features the learned filter consumes. When set
```

```

98 WITHOUT ``classifier_model``, `compare()` trains a fresh model per LOVO fold
99 (honest) ? the recipe, not a pinned artifact.
100 - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
101 overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
102 """
103
104 name: str
105 segmenter: str = "wasb_hybrid"
106 config_overrides: Dict[str, Any] = field(default_factory=dict)
107 cue_flags: Dict[str, bool] = field(default_factory=dict)
108 min_crossings: int = 0
109 min_players: int = 0
110 classifier_model: Optional[str] = None
111 feature_names: Optional[List[str]] = None
112 threshold: float = 0.5
113 merge_gap: float = 1.0
114
115 def is_learned(self) -> bool:
116 """True if this variant applies a learned classifier (pinned model or
117 recipe)."""
118 return self.classifier_model is not None or bool(self.feature_names)
119
120 def to_dict(self) -> dict:
121 return {
122 "name": self.name,
123 "segmenter": self.segmenter,
124 "config_overrides": self.config_overrides,
125 "cue_flags": self.cue_flags,
126 "min_crossings": self.min_crossings,
127 "min_players": self.min_players,
128 "classifier_model": self.classifier_model,
129 "feature_names": self.feature_names,
130 "threshold": self.threshold,
131 "merge_gap": self.merge_gap,
132 }
133
134 @classmethod
135 def from_dict(cls, d: dict) -> "Variant":
136 known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
137 return cls(**{k: v for k, v in d.items() if k in known})
138
139 def spec_hash(self) -> str:
140 """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
141 and makes a result reproducible. The pinned **control** variant always hashes
142 the
143 same, so a regression is always traceable to a recipe change, never to drift."""
144 blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
145 return sha1(blob.encode()).hexdigest()[:12]
146
147 #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
148 #: filter. Always the comparison reference; "rollback" = make this the served variant.
149 CONTROL = Variant(name="control")
150
151 # ----- persisted candidates
152
153
154 @dataclass
155 class VideoCandidates:
156 """One video's persisted detection, ready for offline re-scoring.
157
158 ``intervals`` are the candidate windows; ``features`` is column-major
159 (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
160 ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB

```

```

161 with `load_video_candidates`, or directly in tests.
162 """
163
164 name: str
165 intervals: List[Interval]
166 features: Dict[str, List[float]] = field(default_factory=dict)
167 gts: Optional[List[Interval]] = None
168
169
170 def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
171 """Persisted feature column, defaulting missing/short columns to 0.0 (matches
172 `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
173 col = vc.features.get(name)
174 n = len(vc.intervals)
175 if col is None:
176 logger.warning("variant references feature %r absent from %s ? treating as 0.0",
177 name, vc.name)
178 return [0.0] * n
179 if len(col) != n:
180 raise ExperimentError(
181 f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
182 return [float(x) for x in col]
183
184
185 def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
186 cols = [_feature_column(vc, name) for name in feature_names]
187 return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
188
189
190 def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
191 """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
192 players)."""
193 n = len(vc.intervals)
194 mask = [True] * n
195 if variant.min_crossings > 0:
196 col = _feature_column(vc, "net_crossings")
197 mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
198 if variant.min_players > 0:
199 col = _feature_column(vc, "players_in_court")
200 mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
201 return mask
202
203
204 def predict(variant: Variant, vc: VideoCandidates,
205 model: Optional[LogisticRegression] = None) -> List[Interval]:
206 """Variant prediction: gate by persisted features, optionally filter by a learned
207 model, then merge. Pure and deterministic.
208
209 ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
210 fold; if omitted and the variant pins ``classifier_model``, that frozen model is
211 loaded. A learned variant with neither a supplied nor a pinned model raises ? there
212 is nothing to score with.
213 """
214 mask = _gate_mask(variant, vc)
215 if variant.feature_names:

```

## 12. assistant (2026-06-13T19:15:32.725Z)

This fully confirms the design. The experiment harness's Gate B (`\_gate\_mask`, experiment.py:197-199) also gates on the `players\_in\_court` feature column using `variant.min\_players`, NOT on `in\_court\_ratio`. So in BOTH the served path (`\_select\_for\_handoff`) and the offline harness, `min\_players` thresholds operate on `players\_in\_court`, while `in\_court\_ratio` remains a fixed-definition persisted feature whose internal `min\_players=2` is part of its column definition (documented as "fraction of frames

with 2 in-court players").

I have everything I need. Let me record my assessment.

**\*\*Verification summary:\*\***

Factual claims all hold:

- Two distinct ``min_players`` exist: ``FusionConfig.min_players`` (models.py:102, default 0) and ``compute_person_features`/`in_court_ratio`'s `min_players` (fusion_features.py:200, :99, default 2). CONFIRMED.`
- ``_person_features`` (fusion\_hybrid.py:106-107) calls ``ff.compute_person_features(...)`` without forwarding ``min_players``, so persisted ``in_court_ratio`` always uses the hardcoded default 2, independent of ``FusionConfig.min_players``. CONFIRMED.

But the finding is explicitly NOT a bug:

- The finding itself concedes "No functional bug today" and rates it "low".
- The two are correctly independent **\*\*by design\*\***: ``in_court_ratio`'s threshold-2` is the column's *\*definition\** (docs/MULTI\_SIGNAL\_FUSION\_PLAN.md:138: "fraction of frames with 2 in-court players"), a fixed config-independent feature for P3's learned fusion.
- ``FusionConfig.min_players`` (Gate B) gates on ``players_in_court``, not ``in_court_ratio`` ? verified in both the served path (fusion\_hybrid.py:124) and the offline harness (experiment.py:197-199).
- So the name collision causes no incorrect behavior, no wrong persisted value, no mis-gating.

This is a documentation/readability nitpick about a name collision that the author already correctly handled. The proposed "fix" is an optional one-line comment. It does not represent a defect in the code's behavior. Per the instructions to default to ``real=false`` for nitpicks/non-bugs, this should be marked not-real as a code issue, while acknowledging the underlying facts are accurate.

### 13. user (2026-06-13T19:15:44.714Z)

Structured output provided successfully

### 14. assistant (2026-06-13T19:15:51.284Z)

I've completed the adversarial verification. The finding's factual claims all hold against the code, but it does not constitute a real code issue ? it is a documentation/naming nitpick that the finding itself concedes is "No functional bug today," and I've marked it ``real=false`` accordingly.

Key files verified:

- ``C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/fusion_hybrid.py`` (lines 106-107, 117, 124)
- ``C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/fusion_features.py`` (lines 99, 200, 208)
- ``C:/Users/avidu/Projects/badminton-highlight-indexer/backend/config/models.py`` (line 102)
- ``C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/experiment.py`` (lines 197-199)

The two ``min_players`` are independent **\*\*by design\*\***: the served/offline Gate-B threshold (``FusionConfig.min_players``) gates on the ``players_in_court`` feature column in both the live path and the experiment harness, while ``in_court_ratio`'s internal `min_players=2`` is the fixed *\*definition\** of a config-independent persisted feature column for P3's learned fusion (documented in ``docs/MULTI_SIGNAL_FUSION_PLAN.md:138``). They never interact, so there is no behavioral defect.